

Faculty of Engineering

Internal Design Document

InfluxDB Design for Distributed Real-Time Telemetry System for QUT Microgrid

Date: May 28, 2026

Student Engineer: Dylan Hessing

Student ID: n11245310

Supervisor: Assoc. Prof. Geoff Walker

Version History

Version	Date	Author	Changes / Comments
1	April 21, 2026	Dylan Hessing	Initial Design
2	May 28, 2026	Dylan Hessing	

Contents

1 Overview **3**

2 Scope **3**

3 Architecture Overview **3**

4 InfluxDB Version Selection **4**

5 Schema & Data Model **4**

5.1 Organisation 5

5.2 Buckets 5

5.3 Measurements 5

5.4 Tags 5

5.5 Fields 6

5.6 Timestamp 6

5.7 Representative Schema 6

6 Ingestion Pipeline **7**

6.1 Overview 7

6.2 Telegraf Configuration 7

6.3 Single–Sample Payload Path 7

6.4 Burst Payload Path 8

6.5 Write Batching 8

6.6 QoS Alignment 8

6.7 Error Handling 8

7 Query & Retention Policies **8**

7.1 Retention Design 8

7.2 Downsampling Task 9

7.3 Key Query Parameters 9
7.4 Grafana Integration 10

8 PTP Timestamping Integration 11

8.1 Timestamp Authority 11
8.2 Clock Domain Consistency 11
8.3 Out–of–Order Writes 11
8.4 PTP Status Logging 11
8.5 Timestamp Quality Validation 12

9 Security 12

9.1 Posture 12
9.2 InfluxDB Access Control 12
9.3 Network Binding 12
9.4 Grafana Authentication 13
9.5 Future Hardening 13

10 Deployment & Infrastructure 13

10.1 Host Configuration 13
10.2 Docker Compose 14
10.3 Persistent Volumes 15
10.4 Environment File 15

11 Validation Approach 16

11.1 Objectives 16
11.2 Component-Level Tests 16
11.3 End-to-End Validation 16
11.4 Instrumentation 17

12 Open Questions 17

1 Overview

This document describes the design of the InfluxDB historian comnponent within the distributed microgrid telemetry system. The historian is responsible for receiving time–series electrical measurements from time series electrical measurements from distributed DAQ nodes, persisting with nanosecond–precision timestamps derived from PTP–synchronised STM32 firmware and exposing them for downstream query and analysis.

2 Scope

This document assumes the MQTT protocol design described in the companision document MQTT Protocol Design for Distributed Real-Time Telemetry System for QUT Microgrid (April 2026). In particular, it assumes:

- Mosquitto as the broker
- telegraf as the historian bridge
- The four–level topic hierachy `microgrid/<node_id>/<subsystem>/<measurement>`,
- the JSON payloads carrying a `ts` and `v` field.

Hardware design, DAQ node fabrication and RTOS/firmware decisions are out of scope.

3 Architecture Overview

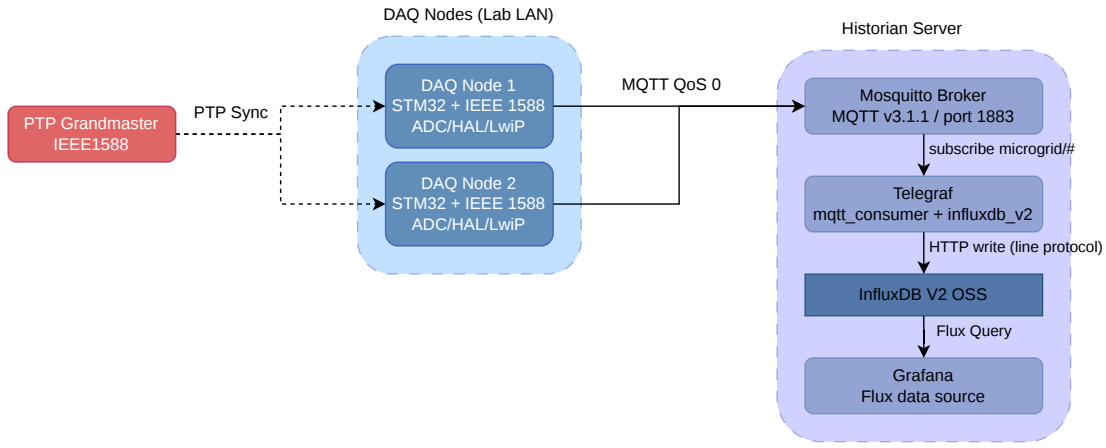


Figure 1

Mosquitto and InfluxDB are co-deployed on the same historian server, consistent with the MQTT design's broker configuration. Telegraf runs on the same server host as the intermediary bridge (no custom subscriber code is required).

4 InfluxDB Version Selection

InfluxDB v2 OSS is selected as the historian for this project. The justification centres on four properties that are directly required by the system design.

Flux query language and native task scheduling. The downsampling design in §7.2 relies on Flux scheduled tasks running aggregateWindow directly within the database, with no external processing component. InfluxDB v2 is the only version for which Flux is the primary, fully–supported query interface. The integrated task scheduler eliminates the need for an external Kapacitor instance, keeping the Docker Compose stack minimal and operationally simple, appropriate for a single–historian research testbed.

Telegraf influxdb_v2 output plugin. The ingestion pipeline is entirely configuration–driven via Telegraf's mqtt_consumer input and influxdb_v2 output plugin. This plugin (including the json_time_key, json_time_format = "unix_ns", and topic_parsing features central to this design) is a first–class, mature integration with InfluxDB v2, with extensive documentation and community usage. No custom subscriber code is required.

Token-based access control. InfluxDB v2's token–based authentication model supports bucket–scoped write tokens, which is the basis of the security posture in §9.2. The Telegraf bridge holds a write–only token scoped to the telemetry bucket; Grafana holds a separate read–only token. This fine–grained token model is native to v2.

Grafana Flux data source. Grafana's native Flux data source for InfluxDB v2 is stable and well–supported, enabling the dashboard panels described in §7.4 to query the historian directly using the same Flux patterns used in the downsampling tasks.

Relationship to InfluxDB 3 Core. InfluxDB 3 Core reached general availability in April 2025. However, InfluxDB 3 Core is not positioned as a direct replacement for InfluxDB OSS v2, and v2 is not planned to reach end–of–life. The v3 architecture is a significant departure — it replaces Flux with native SQL and InfluxQL, and integrates a Python processing engine in place of the Kapacitor/task model used in v2. Migrating to v3 would require rewriting all Flux queries and tasks documented here, with no clear benefit at this testbed scale. InfluxDB v2 OSS therefore remains the appropriate and supported choice for a new deployment of this scope.

Docker image pinning note: The latest tag for InfluxDB Docker images will point to InfluxDB 3 Core from late May 2026 onward. The Compose file in §10.2 uses the explicit influxdb:2 tag to prevent an unintended upgrade during the thesis validation period. This must not be changed to latest.

5 Schema & Data Model

InfluxDB organises data as:

Organisation –> bucket –> Measurement –> Tags + Field + Timestamp

5.1 Organisation

A single InfluxDB organisation `qut-microgrid` is used, consistent with the Telegraf output configuration in the MQTT design document.

5.2 Buckets

Table 1: Buckets

Bucket	Purpose	Retention
`telemetry`	Full-resolution samples from all nodes	90 days (TBD)
`telemetry_1m`	1-minute aggregates for long-term storage	Indefinite

The bucket name `telemetry` matches the Telegraf output configuration. A seperate downsampled bucket avoids data retention growth while preserving long-term trend visibility.

5.3 Measurements

Each MQTT subsystem segment maps to a distinct InfluxDB measurement name:

Table 2: Measuremnts

Measurement	Description
`voltage`	Phase voltage samples (V)
`current`	Phase and neutral current samples (A)
`status`	Node liveness and heartbeat dignostics
`ptp_status`	PTP synchronisation diagnostics

Additional subsystems may be introduced without structural schema changes if the STM32 firmware is extended to publish derived quantities.

5.4 Tags

Tags are string-valued, indexed, and low-cardinality. All three tags are populated automatically by Telegraf's `topic_parsing` from the four-level MQTT topic hierachy.

Table 3: Tags

Tag Key	Source	Example Values	Notes
`node_id`	Topic segment 2	`node_1`, `node_2`	Fixed at firmware flash time, MAC/IP addresses explicitly avoided.
`measurement`	Topic segment 4	`va`, `ib`	Aligned with IEEE-1459-2010 notation

5.5 Fields

Table 4: Tags

Field Key	Type	Source	Units
`v`	float64	JSON payload `v` field	implied by `_measurement` and `measurement` tag (V or A)

5.6 Timestamp

Timestamps are written at nanosecond precision using the `ts` field from the JSON payload, sourced from the STM32 IEEE 1588 hardware timestamping peripheral. The historian does not substitute broker receive time or Telegraf processing time: the node–reported PTP timestamp is authoritative end–to–end.

5.7 Representative Schema

Table 5: Representative Schema

subsystem (_measurement)	node_id	measurement	_field	Example Value
`voltage`	`node_1`	`va`	`v`	
`voltage`	`node_1`	`vb`	`v`	
`voltage`	`node_1`	`vc`	`v`	
`current`	`node_1`	`ia`	`v`	
`current`	`node_1`	`ib`	`v`	
`current`	`node_1`	`ic`	`v`	
`ptp_status`	`node_1`	`offset_ns`	`v`	

6 Ingestion Pipeline

6.1 Overview

The ingestion pipeline is fully configuration–driven via Telegraf's MQTT consumer input plugin and InfluxDB v2 output plugin. No custom subscriber code is required for the single–sample payload path. Telegraf is co–deployed on the historian server alongside the Mosquitto and InfluxDB.

6.2 Telegraf Configuration

```
[[inputs.mqtt_consumer]]
  servers     = ["tcp://localhost:1883"]
  topics      = ["microgrid/#"]
  username    = "telegraf"
  password    = "${TELEGRAF_MQTT_PASSWORD}"
  qos         = 0
  data_format = "json"
  json_time_key    = "ts"
  json_time_format = "unix_ns"

  [[inputs.mqtt_consumer.topic_parsing]]
    topic       = "microgrid/<node_id>/<subsystem>/<measurement>"
    tags        = "_/<node_id>/_/<measurement>"
    measurement = "_/_/<subsystem>/_"

[[outputs.influxdb_v2]]
  urls         = ["http://localhost:8086"]
  token        = "${INFLUXDB_TOKEN}"
  organization = "qut-microgrid"
  bucket       = "telemetry"
```

Note:

- `json_time_key "ts"` and `json_time_format "unix_ns"` instructs Telegraph to use the STM32 PTP timestamp as the InfluxDB point timestamp verbatim. Telegraf processing trime is never substituted.
- `topic_parsing` extracts `node_id` and `measurement` as tags. The `subsystem` segment drives the InfluxDB measurement name via the `measurement` key. The leading `microgrid` segment is ignored.
- Credentials are injected via environment variables and are never stored in the config files, consistent with the MQTT design security posture.

6.3 Single–Sample Payload Path

For the single-sample payload `{"ts": <unix_ns>, "v": <float>}`, Telegraf requires no transformation. The `v` field maps directly to the `_field` column and the timestamp is extracted from `ts`. This path is

fully supported by the native Telegraf JSON parser with zero custom logic.

6.4 Burst Payload Path

The burst payload format (`{"ts": <unix_ns>, "dt": <seconds>, "v": [...]}`), defined in the MQTT design, is not natively handled by Telegraf's JSON parser: it cannot reconstruct per-sample timestamps from a `ts` + `dt` + array structure.

6.5 Write Batching

Table 6: Write Batching

Parameter	Default	Notes
`metric_batch_size`	1000 points	Maximum points per write request
`flush_interval`	10s	Maximum time between flushes

For a two-node, 6–8 channel testbed at modest sample rates these defaults are conservative. If near-real-time dashboard visibility is required, flush_interval should be reduced to 1s. This is a runtime tuning decision and does not affect the schema or pipeline architecture.

6.6 QoS Alignment

Telemetry samples are published at QoS 0, consistent with the MQTT design. The ingestion pipeline makes no attempt to detect or fill gaps in the historian record resulting from dropped packets or node disconnection. Gaps are visible in the historian and are acceptable for a research testbed.

6.7 Error Handling

Telegraf drops malformed payloads that fail JSON parsing and logs an error. No dead-letter queue is implemented. This is acceptable given:

- The two-node testbed operates on a wired LAN with low packet loss probability
- Telemetry samples are QoS 0 by design — loss is already tolerated at the transport layer
- A dead-letter mechanism would add operational complexity disproportionate to the research context

7 Query & Retention Policies

7.1 Retention Design

Retention is configured at the bucket level in InfluxDB v2. No retention policy is applied within a bucket; the bucket-level setting is the sole retention mechanism.

Table 7: Measuremnts

Bucket	Telemetry	Rationale
`telemetry`	90 days	Covers the active thesis validation period at full resolution
`telemetry_1m`	Indefinite	Long-term aggregate data for reporting and trend analysis

7.2 Downsampling Task

A Flux scheduled task aggregates telemetry into telemetry_1m at 1-minute intervals. mean, min, and max are stored as separate fields to support both trend analysis and power quality event detection within each window.

```
option task = {name: "downsample_1m", every: 1m}

from(bucket: "telemetry")
  |> range(start: -task.lastSuccessTime)
  |> filter(fn: (r) => r._field == "v")
  |> aggregateWindow(every: 1m, fn: mean, createEmpty: false)
  |> set(key: "_field", value: "v_mean")
  |> to(bucket: "telemetry_1m")

from(bucket: "telemetry")
  |> range(start: -task.lastSuccessTime)
  |> filter(fn: (r) => r._field == "v")
  |> aggregateWindow(every: 1m, fn: min, createEmpty: false)
  |> set(key: "_field", value: "v_min")
  |> to(bucket: "telemetry_1m")

from(bucket: "telemetry")
  |> range(start: -task.lastSuccessTime)
  |> filter(fn: (r) => r._field == "v")
  |> aggregateWindow(every: 1m, fn: max, createEmpty: false)
  |> set(key: "_field", value: "v_max")
  |> to(bucket: "telemetry_1m")
```

7.3 Key Query Parameters

The following Flux examples cover the primary query patterns anticipated during validation.

Time-window data for a single channel:

```
from(bucket: "telemetry")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "voltage"
                    and r.node_id == "node_1"
                    and r.measurement == "va")
```

Cross-node comparison for the same channel:

```
from(bucket: "telemetry")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "voltage"
                    and r.measurement == "va")
  |> group(columns: ["node_id"])
```

Cross–node comparisons are only meaningful where PTP synchronisation is confirmed locked. The ptp_status measurement should be checked before drawing cross–node conclusions from the historian.

All channels from a single node (debugging):

```
from(bucket: "telemetry")
  |> range(start: -15m)
  |> filter(fn: (r) => r.node_id == "node_1")
```

PTP synchronisation quality over a window:

```
from(bucket: "telemetry")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "ptp_status"
                    and r.node_id == "node_1")
```

7.4 Grafana Integration

Grafana is a confirmed thesis deliverable. Grafana connects to InfluxDB v2 via the native Flux data source. Key dashboard panels anticipated:

Table 8: Measuremnts

Panel	Source Bucket	Query Pattern
Per–phase voltage time–series	telemetry	Single–Channel, cross–node
Per–phase current time–series	telemetry	Single–Channel, cross–node
PTP offset over time (per node)	telemetry	ptp_status measurement
1–minute voltage mean/min/max	telemetry_1m	Single–Channel, cross–node

Grafana dashboard design is out of scope for this document and will be addressed in a separate deliverable.

8 PTP Timestamping Integration

8.1 Timestamp Authority

The STM32 nodes are the authoritative timestamp source. Each node disciplines its local clock to the PTP grandmaster prior to sampling. The `ts` field in the MQTT payload reflects the time of ADC sample capture via the IEEE 1588 hardware timestamping peripheral;not broker receive time or Telegraf processing time.

Telegraf's `json_time_key` and `json_time_format = "unix_ns"` configuration ensures this timestamp is used as the InfluxDB timestamp verbatim.

8.2 Clock Domain Consistency

All nodes share a single PTP clock domain. The historian server's system clock must also be synchronised to the same grandmaster, or to a sufficiently accurate NTP source, to ensure that Flux query time ranges expressed in wall-clock time correspond correctly to stored point timestamps.

8.3 Out–of–Order Writes

MQTT QoS 0 with no ordering guarantees means Telegraf may deliver points to InfluxDB with timestamps older than recently written points. For example, following a node reconnection where buffered status messages are flushed. InfluxDB handles out–of–order writes within a TSM shard natively. Default shard duration in InfluxDB v2 is one week; behaviour at shard boundaries should be verified if a node reconnects after an outage longer than one week, though this is unlikely in a lab testbed context.

8.4 PTP Status Logging

Each DAQ node publishes PTP diagnostic data periodically to `microgrid/<node_id>/ptp_status/offset_ns` (and related topics), stored in the `ptp_status` measurement in InfluxDB. This allows post–hoc assessment of timestamp quality for any given data window — a prerequisite for validating cross–node comparisons drawn from the historian.

The fields stored under the `ptp_status` subsystem are:

Table 9: PTP_status subsystem fields

measurement tag	field v meaning	Unit
`offset_ns`	Clock offset from PTP grandmaster	ns
`path_delay_ns`	Mean path delay to grandmaseet	ns
`sync_state`	Synchronisation state	(LOCKED, ACQUIRING, HOLDOVER)

8.5 Timestamp Quality Validation

Before cross-node comparisons are drawn from the historian, the following conditions should be verified from the `ptp_status` measurement:

- `sync_state` is LOCKED for all nodes across the relevant time window
- `offset_ns` is stable and within the sub–microsecond range expected on a switched Ethernet LAN
- No `HOLDOVER` periods are present, which would indicate the node was operating on a free–running clock

This validation is a prerequisite for any result in the thesis that relies on cross–node temporal alignment.

9 Security

9.1 Posture

The historian operates on a university research lab LAN and is not exposed to the public internet. The security design is therefore pragmatic and aligned with the phased approach documented in the companion MQTT design document (§7.3). This section describes historian-side measures only; broker-side authentication, ACLs, and TLS are covered in the MQTT design document.

9.2 InfluxDB Access Control

InfluxDB v2 uses token-based authentication for all API access. Two tokens are issued:

Table 10: InfluxDB API Tokens

Token	Scope	Used by
Telegraf write token	Write–only to `telemetry` bucket	Telegraf bridge
Admin token	Full organisation access	Local admin / ad–hoc Flux queries

The Telegraf write token is scoped to the minimum required permission: write access to the `telemetry` bucket only. It cannot read data, access other buckets, or modify organisation configuration. This limits the blast radius if the token is compromised.

Tokens are stored as environment variables and injected into the Telegraf configuration at runtime via `${INFLUXDB_TOKEN}`. They are never written to configuration files or committed to version control.

9.3 Network Binding

The InfluxDB HTTP API is bound to loopback (`localhost:8086`) only and is not reachable from the lab LAN directly. All access from Telegraf and Grafana is local to the historian server. This is enforced at the InfluxDB configuration level:

```
# /etc/influxdb/config.toml
http--bind--address = "127.0.0.1:8086"
```

9.4 Grafana Authentication

Grafana is deployed with local authentication enabled. A minimum of two accounts are provisioned:

Table 11: Grafana User Accounts

Account	Role	Purpose
`admin`	Admin	Dashboard and data source configuration
`viewer`	Viewer	Read–only dashboard access for demonstration

Anonymous access is disabled. The Grafana InfluxDB data source is configured with a dedicated read–only token. It does not use the admin token.

9.5 Future Hardening

Consistent with the MQTT design's phased security model, the following measures are deferred but noted:

Table 12: Deferred Security Measures

Measure	Trigger
TLS on InfluxDB API (`https://`)	If historian is accessed from outside the lab VLAN
Grafana OAuth / SSO	If multi–user access beyond the research team is required
InfluxDB audit logging	If data integrity evidence is required for thesis examination

10 Deployment & Infrastructure

10.1 Host Configuration

All components (Mosquitto, Telegraf, InfluxDB, and Grafana) are co–deployed on a single historian server on the lab LAN. Co–deployment is justified at the two–node testbed scale; resource contention between components is not anticipated.

Table 13: Component Deployment Summary

Component	Version	Deployment method
InfluxDB	v2.x OSS	Docker container
Telegraf	Latest stable	Docker container
Mosquitto	2.x	Docker container
Grafana	Latest stable	Docker container

10.2 Docker Compose

All components are managed via a single Docker Compose file. This ensures reproducible deployment, simplifies inter–service networking, and allows the full stack to be brought up or torn down with a single command, appropriate for a research testbed where the environment may need to be rebuilt.

```
version: "3.8"

services:

  mosquitto:
    image: eclipse-mosquitto:2
    container_name: mosquitto
    ports:
      - "127.0.0.1:1883:1883"
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log
    restart: unless-stopped

  influxdb:
    image: influxdb:2
    container_name: influxdb
    ports:
      - "127.0.0.1:8086:8086"
    volumes:
      - influxdb-data:/var/lib/influxdb2
      - ./influxdb/config:/etc/influxdb2
    environment:
      - DOCKER_INFLUXDB_INIT_MODE=setup
      - DOCKER_INFLUXDB_INIT_USERNAME=${INFLUXDB_ADMIN_USER}
      - DOCKER_INFLUXDB_INIT_PASSWORD=${INFLUXDB_ADMIN_PASSWORD}
      - DOCKER_INFLUXDB_INIT_ORG=qut-microgrid
      - DOCKER_INFLUXDB_INIT_BUCKET=telemetry
      - DOCKER_INFLUXDB_INIT_RETENTION=90d
    restart: unless-stopped

  telegraf:
    image: telegraf:latest
    container_name: telegraf
    depends_on:
      - mosquitto
      - influxdb
    volumes:
      - ./telegraf/telegraf.conf:/etc/telegraf/telegraf.conf:ro
    environment:
      - TELEGRAF_MQTT_PASSWORD=${TELEGRAF_MQTT_PASSWORD}
      - INFLUXDB_TOKEN=${INFLUXDB_TOKEN}
```

```
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "127.0.0.1:3000:3000"
    volumes:
      - grafana-data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_ADMIN_PASSWORD}
      - GF_AUTH_ANONYMOUS_ENABLED=false
    restart: unless-stopped

volumes:
  influxdb-data:
  grafana-data:
```

All ports are bound to `127.0.0.1` only, consistent with the network binding posture in §7.3. Sensitive values are passed via environment variables sourced from a `.env` file that is excluded from version control.

10.3 Persistent Volumes

InfluxDB and Grafana state are stored in named Docker volumes. Mosquitto persistence (QoS 1 message store) is bind-mounted to a host directory for visibility. The Telegraf configuration is bind-mounted read-only.

10.4 Environment File

A `.env` file in the Compose project root holds all secrets:

```
INFLUXDB_ADMIN_USER=admin
INFLUXDB_ADMIN_PASSWORD=<redacted>
INFLUXDB_TOKEN=<redacted>
TELEGRAF_MQTT_PASSWORD=<redacted>
GRAFANA_ADMIN_PASSWORD=<redacted>
```

This file must not be committed to version control. A `.env.example` file with placeholder values should be committed in its place.

11 Validation Approach

11.1 Objectives

The validation approach verifies that the historian correctly receives, stores, and exposes PTP-timestamped measurements from the DAQ nodes. Three properties are of primary interest:

- **Correctness**: data written to InfluxDB matches the values published by the STM32 nodes
- **Timestamp fidelity**: the PTP timestamp from the STM32 is preserved end-to-end with no substitution or modification
- **Cross-node alignment**: points from two synchronised nodes at the same physical instant share consistent timestamps within the expected PTP accuracy bound

11.2 Component-Level Tests

Before end-to-end validation, each pipeline stage is verified in isolation.

Table 14: Component-Level Validation Tests

Test	Method	Pass condition
Telegraf MQTT subscription	Publish a known payload with `mosquitto_pub`; inspect Telegraf logs	Payload parsed, point written to `telemetry` bucket
Timestamp preservation	Publish payload with known `ts` value; query InfluxDB and compare	Stored timestamp matches `ts` to nanosecond precision
Tag extraction	Publish to a known topic; query tags on the stored point	`node_id` and `measurement` tags match topic segments
Subsystem routing	Publish to `voltage` and `current` topics; query both measurements	Points land in correct InfluxDB measurements
Downsampling task	Write synthetic data spanning $>$1 minute; trigger task manually	`telemetry_1m` contains `v_mean`, `v_min`, `v_max` for the window

11.3 End-to-End Validation

End-to-end validation uses the two-node testbed with both STM32 nodes publishing live measurements.

Timestamp fidelity test: A known electrical event (e.g. a step load change) is applied to the microgrid. The timestamp of the event as recorded in the historian for each node is compared. The difference should be within the expected PTP synchronisation accuracy (sub-microsecond on a switched Ethernet LAN). A large discrepancy indicates either a timestamp substitution error in the pipeline or a PTP synchronisation failure.

Data completeness test: Both nodes publish at a known rate for a fixed duration. The number of points in the historian for each channel is compared against the expected count. Any gap is investigated against Telegraf and Mosquitto logs.

PTP status logging test: The `ptp_status` measurement is queried over a sustained period. The `sync_state` field should remain `LOCKED` throughout normal operation, with `offset_ns` stable within the sub-microsecond range.

11.4 Instrumentation

During validation, Telegraf's `influxdb_internal` input plugin is enabled to expose pipeline metrics (including points received, write errors, and flush latency) into a separate `telegraf_internal` bucket. This provides visibility into ingestion health without relying solely on application logs.

```
[[inputs.internal]]
  collect_memstats = false

[[outputs.influxdb_v2]]
  urls         = ["http://localhost:8086"]
  token        = "${INFLUXDB_TOKEN}"
  organization = "qut-microgrid"
  bucket       = "telegraf_internal"
```

12 Open Questions

The following items remain unresolved at the time of writing and must be closed before the implementation phase begins.

Table 15: Open Questions & Next Steps

#	Question	Impact	Owner
1	ADC sample rate per channel	Drives storage volume estimate, determines whether burst payloads are needed, and sets `flush_interval` tuning requirement	Dylan / Ryan
2	Single-sample vs burst payload	Determines whether burst pipeline path must be implemented	Dylan / Ryan
3	TAI vs UTC in STM32 PTP stack	Must match historian server time source; epoch mismatch silently corrupts query time ranges	Dylan / Ryan
4	Grafana on same host or separate	Determines whether `localhost` InfluxDB binding holds or must be relaxed	Dylan
5	`sync_state` as string or integer field	Affects Flux query complexity for PTP status panels in Grafana	Dylan
6	`min`/`max` downsampling required	Determines whether downsampling task requires three passes or one	Dylan / Geoff
7	Historian server hardware confirmed	Confirms Docker Compose deployment is viable; rules out resource contention	Dylan
8	90-day raw retention vs disk capacity	Prevents historian running out of storage during thesis validation period	Dylan